Innovation Centre for Education

YENEPOYA INSTITUTE OF ARTS, SCIENCE , COMMERCE AND MANAGEMENT

HLD on Project : Lightweight Finance API

Student Name: Sayooj.V

Industry Mentor: Mr. Sumit K Shukla

Guided by:

Table of Contents

---

1. Introduction

1.1. Scope of the document

The scope of this document is to present the high-level architectural design of the Lightweight Finance API system. It provides an overview of the system structure, major components, their interactions, and the overall data flow from client request to server response.

This document focuses on system-level design aspects including architecture patterns, component responsibilities, and technology choices. It does not cover detailed implementation logic, algorithms, or code-level specifications, which are addressed in the Low-Level Design (LLD) document.

The document serves as a reference for developers, reviewers, and stakeholders to understand how the system is organized and how different parts of the application work together.

1.2. Intended Audience

This document is primarily intended for academic evaluators, university project guides, industry mentors, and prospective developers seeking to understand the structural logic behind the application.

1.3. System overview

The Lightweight Finance API is a REST-based application built using FastAPI to manage financial data such as income, expenses, and budgets. It provides HTTP endpoints to add data and retrieve summaries in JSON format.

The system follows a layered monolithic architecture with clear separation between API, business logic, and data layers. It is designed to be stateless, ensuring each request is processed independently. In the current version, data is stored in-memory, with support for future scalability and database integration.

2. System Design

2.1. Application Design

The application is designed using a layered monolithic architecture, where all components are part of a single deployable unit. The system is structured into multiple layers including the client layer, API layer, business logic layer, and data layer.

The API layer is built using FastAPI and handles HTTP requests and responses. The business logic layer processes financial operations, while the data layer manages in-memory storage. The application follows REST principles and uses a stateless request-response model with JSON data exchange.

2.2. Process Flow

1)The client sends an HTTP request to the API with required data in JSON format.

2)The FastAPI server receives the request and routes it to the appropriate endpoint.

3)Input data is validated using Pydantic models.

4)If validation is successful, the request is passed to the business logic layer.

5)The business logic processes the request and performs required operations.

6)The system stores or retrieves data from the in-memory data layer.

7)The processed result is returned through the API layer.

8)The client receives the response in JSON format.

9)**If validation fails, an error response is returned to the client.**

2.3. Information Flow

The information flow in the system follows a layered approach where data moves from the client to the API layer as JSON requests, then to the business logic layer for processing, and finally to the data layer for storage or retrieval. The processed data is returned back through the same layers in reverse order as a JSON response. Each layer interacts only with its adjacent layer, ensuring a clear and controlled flow of information.

2.4. Components Design

Client Module: Sends requests and receives responses.

API Module: Handles routing, validation, and responses using FastAPI.

Business Logic Module: Processes financial operations.

Data Module: Stores data in memory during runtime.

Container Module: Runs the application using Docker.

2.5. Key Design Considerations

Separation of Concerns: The system is divided into layers to keep responsibilities clear and manageable.

Stateless Design: Each request is independent and does not rely on previous interactions.
RESTful Approach: Uses standard HTTP methods and JSON-based communication.

Simplicity: Monolithic design keeps development and testing straightforward.

Scalability: Designed to support future upgrades like database integration and cloud deployment.

2.6. API Catalogue

1)POST /income: Adds income data
2)POST /expense: Adds expense data
3)POST /budget: Sets budget limit
4)GET /summary: Retrieves financial summary
5)GET /budget: Checks budget status

---

3. Data Design

3.1. Data Model

The data model consists of simple in-memory structures where income and expense values are stored as lists of numbers, and the budget is maintained as a single numeric value. These data elements are used to perform financial calculations such as totals and balance within the application.

3.2. Data Access Mechanism

Data is accessed through the business logic layer, where the application reads and writes information to in-memory data structures. All operations are performed via API endpoints, ensuring that the client interacts only with the API layer and not directly with the data storage.

3.3. Data Retention Policies

Data is accessed through the business logic layer, where the application reads and writes information to in-memory data structures. All operations are performed via API endpoints, ensuring that the client interacts only with the API layer and not directly with the data storage.

3.4. Data Migration

Currently, no data migration is implemented as the system uses in-memory storage. In future versions, data can be migrated to a persistent database such as PostgreSQL, with necessary updates to the data and business logic layers.

---

4. Interfaces

The system interfaces are based on RESTful APIs, where the client interacts with the application through HTTP endpoints. All requests and responses are in JSON format. Tools such as Postman and Swagger UI are used to test and access these interfaces.

---

5. State and Session Management

The system follows a stateless design, where no session data is stored on the server between requests. Each request is processed independently, and all required information is provided within the request itself.

---

6. Caching

The current system does not implement any caching mechanism, as it uses in-memory data storage and handles a limited scope of operations.

---

7. Non-Functional Requirements

7.1. Security Aspects

The system ensures basic security through input validation using Pydantic, which checks data types and value ranges. Proper error handling is implemented to prevent i nvalid data processing. Currently, authentication and advanced security features are n ot included and are planned for future versions.

7.2. Performance Aspects

The system provides good performance by using FastAPI with Uvicorn for fast request handling. In-memory data storage allows quick data access, and the stateless design ensures efficient processing of each request. This setup is suitable for small-scale and demo-level usage.

---

8. References

DOC-01: Project Proposal

DOC-02: Product Requirements Document (PRD)

DOC-04: Low-Level Design (LLD)

DOC-05: API Reference